Working with Text Data in Pandas

Agenda

- Text Data in Pandas
- Iteration
  - Iterating over Rows
  - Applying a Function to Each Element
  - Vectorized Operations
  - Iterating over Series
- Sorting
  - Sorting DataFrame by Column
  - Sorting DataFrame by Multiple Columns
  - Sorting DataFrame by Index
  - Sorting a Series
- Plotting with Pandas

1. Text Data in Pandas

Working with text data in Pandas involves various operations to manipulate and analyze textual information. Here are some common text data operations using Pandas:

```
# Assuming 'df' is DataFrame with a 'Column' containing text data
import pandas as pd

df = pd.DataFrame({'Column': ['Hello', 'World', 'Python', 'Data Science']})

# Calculates the length of each string
df['Length'] = df['Column'].str.len()
print("Length of each string:")
print(df[['Column', 'Length']])
```

```
Length of each string:
         Column  Length
0         Hello       5
1         World       5
2        Python       6
3  Data Science      12
```

```
# Assuming 'df' is DataFrame with a 'Column' containing text data
df = pd.DataFrame({'Column': ['Hello', 'World', 'Python', 'Data Science']})

# Converts text to lowercase
df['Lowercase'] = df['Column'].str.lower()
print("\nText in lowercase:")
print(df[['Column', 'Lowercase']])
```

```
Text in lowercase:
         Column     Lowercase
0         Hello         hello
1         World         world
2        Python        python
3  Data Science  data science
```

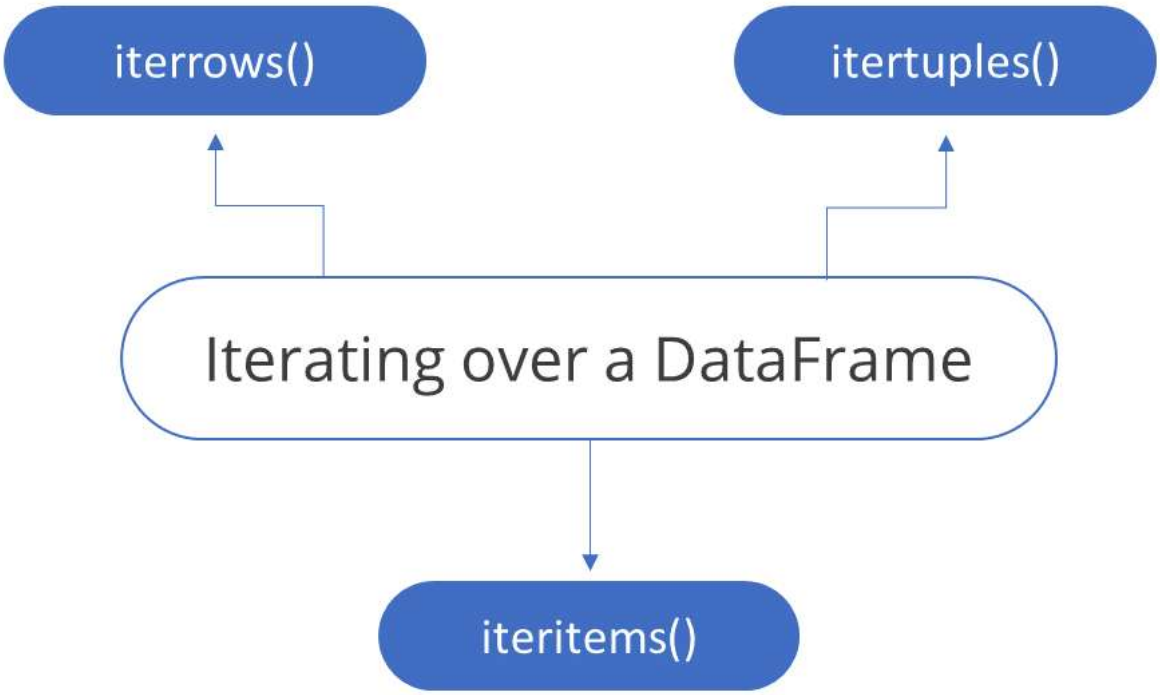
```
# Assuming 'df' is DataFrame with a 'Column' containing text data
df = pd.DataFrame({'Column': ['Hello', 'World', 'Python', 'Data Science']})

# Checks if each string contains the specified substring
substring = 'Data'
df['ContainsSubstring'] = df['Column'].str.contains(substring)
print("\nContains substring 'Data':")
print(df[['Column', 'ContainsSubstring']])
```

```
Contains substring 'Data':
         Column  ContainsSubstring
0         Hello              False
1         World              False
2        Python              False
3  Data Science               True
```

2. Iteration

Iteration in Pandas typically involves traversing through the rows or elements of a DataFrame or Series.

- However, it is important to note that direct iteration over DataFrame rows using Python's for loop is generally discouraged due to performance reasons.
- Instead, Pandas provides efficient methods for iteration and applying functions to DataFrame elements.



2.1 Iterating over Rows

```
import pandas as pd

# Assuming 'df' is your DataFrame with columns 'Column1' and 'Column2'
df = pd.DataFrame({'Column1': [1, 2, 3], 'Column2': ['A', 'B', 'C']})

for index, row in df.iterrows():
    print(f"Index: {index}, Data: {row['Column1']}, {row['Column2']}")
```

```
Index: 0, Data: 1, A
Index: 1, Data: 2, B
Index: 2, Data: 3, C
```

2.2 Applying a Function to Each Element

```
# Assuming 'df' is your DataFrame with 'ExistingColumn'
df = pd.DataFrame({'ExistingColumn': [10, 20, 30]})

df['NewColumn'] = df['ExistingColumn'].apply(lambda x: x * 2)
print(df)
```

```
   ExistingColumn  NewColumn
0              10         20
1              20         40
2              30         60
```

2.3 Vectorized Operations

```
# Assuming 'df' is your DataFrame with 'ColumnA' and 'ColumnB'
df = pd.DataFrame({'ColumnA': [1, 2, 3], 'ColumnB': [4, 5, 6]})

df['ResultColumn'] = df['ColumnA'] + df['ColumnB']
print(df)
```

```
   ColumnA  ColumnB  ResultColumn
0        1        4             5
1        2        5             7
2        3        6             9
```

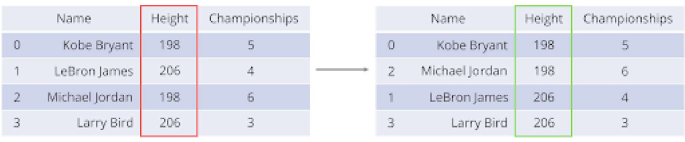
2.4 Iterating over Series

```
# Assuming 'series' is your Pandas Series
series = pd.Series([10, 20, 30], name='Values')

for index, value in series.items():
    print(f"Index: {index}, Value: {value}")
```

```
Index: 0, Value: 10
Index: 1, Value: 20
Index: 2, Value: 30
```

3. Sorting

Sorting in Pandas involves arranging the elements of a DataFrame or Series based on specific criteria, such as column values or indices.



	Name	Height	Championships
0	Kobe Bryant	198	5
1	LeBron James	206	4
2	Michael Jordan	198	6
3	Larry Bird	206	3

	Name	Height	Championships
0	Kobe Bryant	198	5
2	Michael Jordan	198	6
1	LeBron James	206	4
3	Larry Bird	206	3

3.1 Sorting DataFrame by Column

```
# Create a sample DataFrame
df = pd.DataFrame({'Name': ['Alice', 'Bob', 'Charlie'],
                   'Age': [25, 22, 30],
                   'Salary': [50000, 60000, 75000]})

# Sort DataFrame by the 'Age' column in ascending order
df_sorted = df.sort_values(by='Age')
print("Sorted DataFrame by Age:\n", df_sorted)
```

```
Sorted DataFrame by Age:
       Name  Age  Salary
1      Bob   22   60000
0    Alice   25   50000
2  Charlie   30   75000
```

3.2 Sorting DataFrame by Multiple Columns

```
# Sort DataFrame by 'Age' in ascending order, then by 'Salary' in descending order
df_sorted_multi = df.sort_values(by=['Age', 'Salary'], ascending=[True, False])
print("\nSorted DataFrame by Age and Salary:\n", df_sorted_multi)
```

```
Sorted DataFrame by Age and Salary:
       Name  Age  Salary
1      Bob   22   60000
0    Alice   25   50000
2  Charlie   30   75000
```

3.3 Sorting DataFrame by Index

```
# Sort DataFrame by index in descending order
df_sorted_index = df.sort_index(ascending=False)
print("\nSorted DataFrame by Index:\n", df_sorted_index)
```

```
Sorted DataFrame by Index:
       Name  Age  Salary
2  Charlie   30   75000
1      Bob   22   60000
0    Alice   25   50000
```

3.4 Sorting a Series

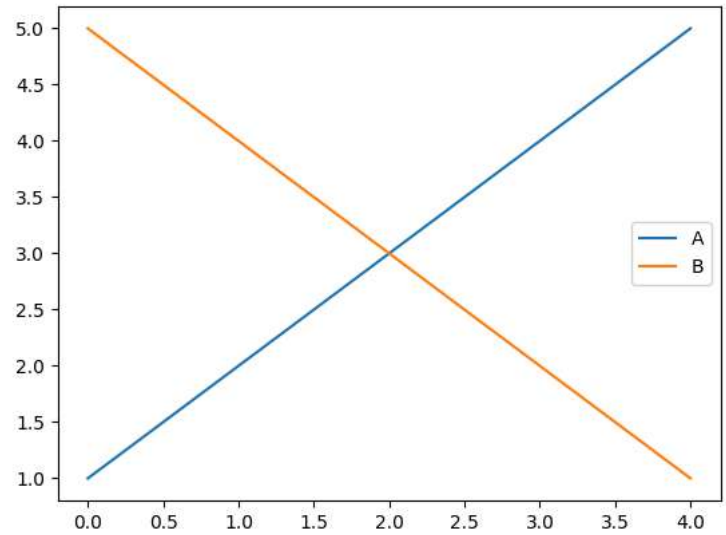
```
# Create a sample Series
series = pd.Series([25, 22, 30], index=['Alice', 'Bob', 'Charlie'], name='Age')

# Sort Series in descending order
series_sorted = series.sort_values(ascending=False)
print("\nSorted Series by Age:\n", series_sorted)
```

```
Sorted Series by Age:
 Charlie    30
Alice      25
Bob        22
Name: Age, dtype: int64
```

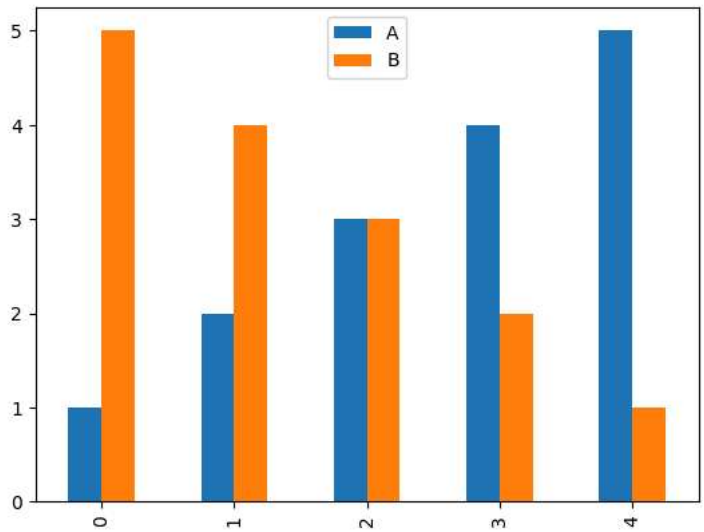
4. Plotting with Pandas

Plotting data is required to visualize the data in Python. Python uses the plot method in pandas to create plots; there are many types of plots available in pandas.

```
import pandas as pd
import matplotlib.pyplot as plt

# Create a sample DataFrame
data = {'A': [1, 2, 3, 4, 5], 'B': [5, 4, 3, 2, 1]}
df = pd.DataFrame(data)

# Plot a line chart
df.plot()
plt.show()
```




```
# Plot a bar chart
df.plot(kind='bar')
plt.show()
```



Assisted Practice

Problem Statement:

Create a detailed report on the monthly weather data by performing text manipulation, data sorting, and visualization to analyze temperature and precipitation trends.

Data: The dataset contains daily observations of temperature and precipitation over a month.

Steps to Perform:

1. Textual manipulation

- Convert 'Day' to a string format with appropriate suffixes (1st, 2nd, 3rd, and so on)
- Classify 'Temperature' into categories (Low, Medium, High) based on predefined thresholds
- Determine if 'Precipitation' falls under 'Light', 'Moderate', or 'Heavy' rainfall

2. Iteration and data aggregation

- Iterate over the DataFrame to calculate weekly averages of temperature and precipitation
- Summarize findings in a new DataFrame

3. Sorting

- Sort the DataFrame by 'Temperature' and 'Precipitation' in ascending and descending order
- Sort the DataFrame by the day of the month

4. Plotting

- Create a line plot for temperature trends
- Create a bar plot to compare precipitation levels across the month